SPM: Message Passing Interface (MPI) (Part II)

Academic year 2025-2026

Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

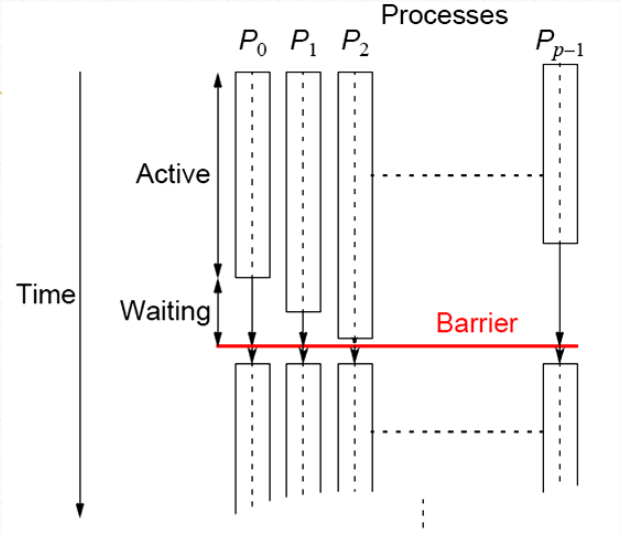
- Collective communications
- Simple usage examples

Collective Communication in MPI

- Multi-process communication patterns
  - Examples: one-to-all, all-to-one, all-to-all
- Often more efficient than implementing the same pattern with separate point-to-point communications
  - From a functional standpoint, collectives are not necessary. They can be simulated with point-to-point communications
- MPI provides an extensive set of collective operations
- All processes in a communicator must call the same collective operation

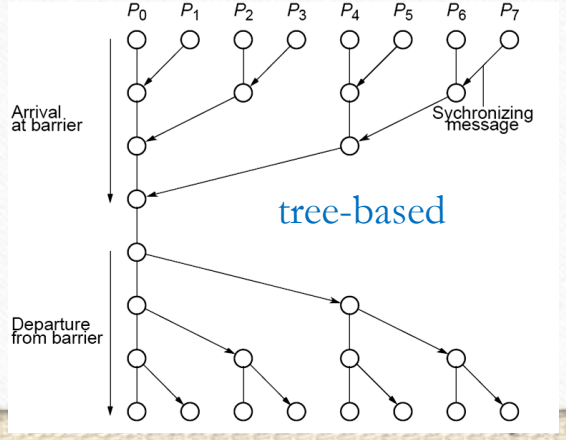
Barrier

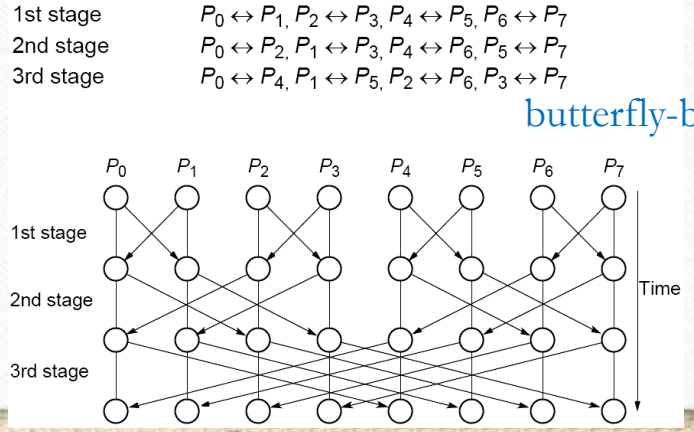
- int MPI_Barrier(MPI_Comm comm)
  - Wait until all processes in the communicator reach the barrier
- int MPI_Ibarrier(MPI_Comm comm, MPI_Request* req)
  - Non-blocking version. All collectives have a non-blocking version
  - Completion must be checked with MPI_Wait/MPI_Test on the returned request
- Possible implementations:
  - A process enters the arrival phase and does not leave until all others have arrived. Then they move to the departure phase




```
      // Master process (P0)
for(i=1; i < p; ++i) recv(PANY);
for(i=1; i < p; ++i) send(Pi);
      // Slave processes (P1,…,Pp)
send(P0);
recv(P0);
```

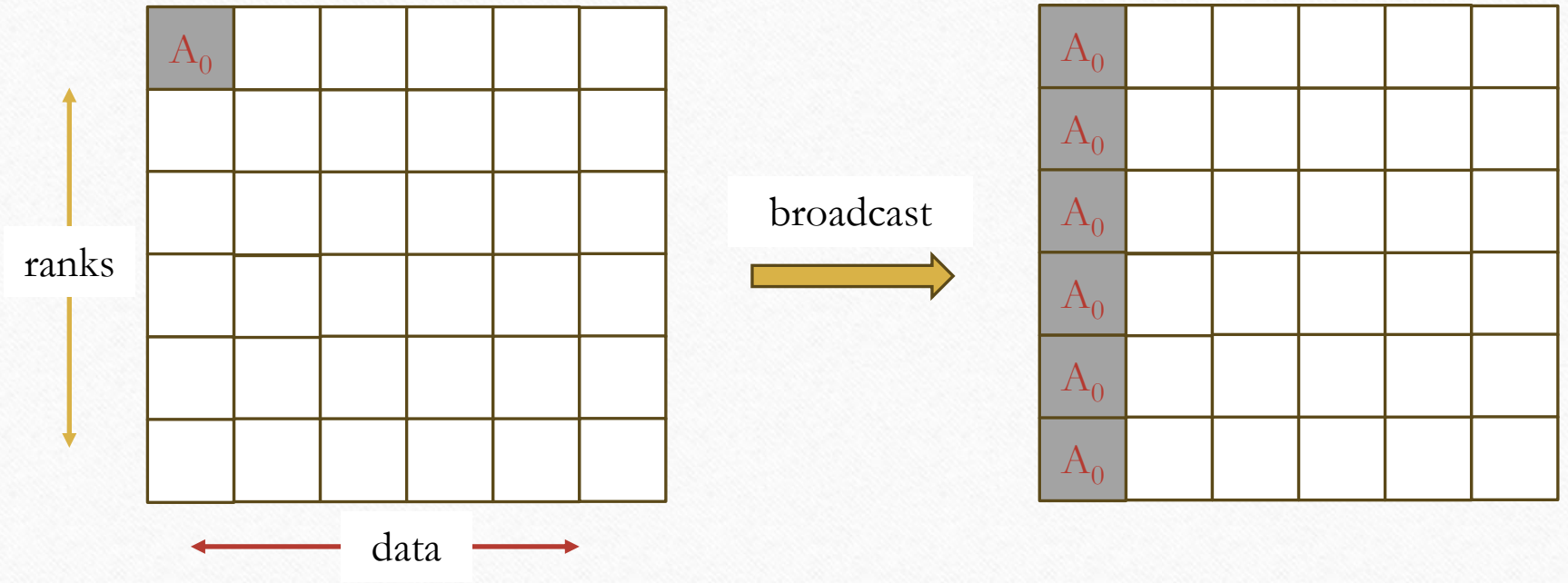
centralized implementation





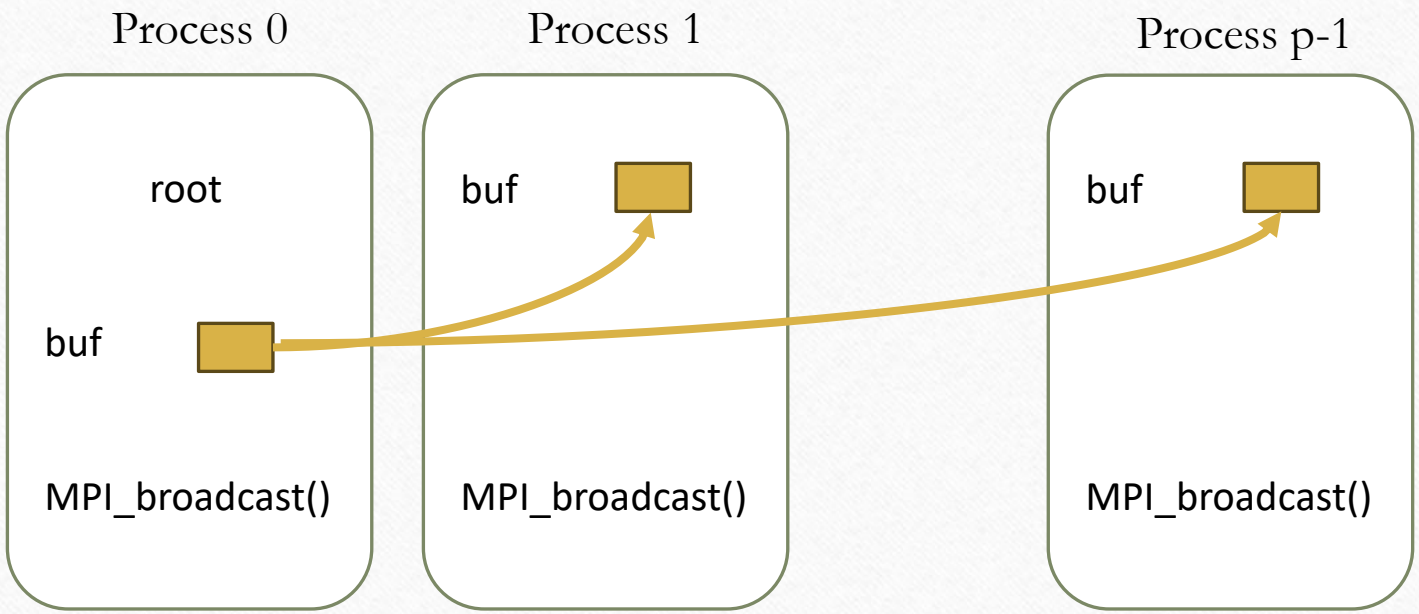
Broadcast

- int MPI_Bcast(void* buf, int count, MPI_Datatype dt, int root, MPI_Comm comm)
  - Sends the same message to all processes in the communicator (one-to-all)



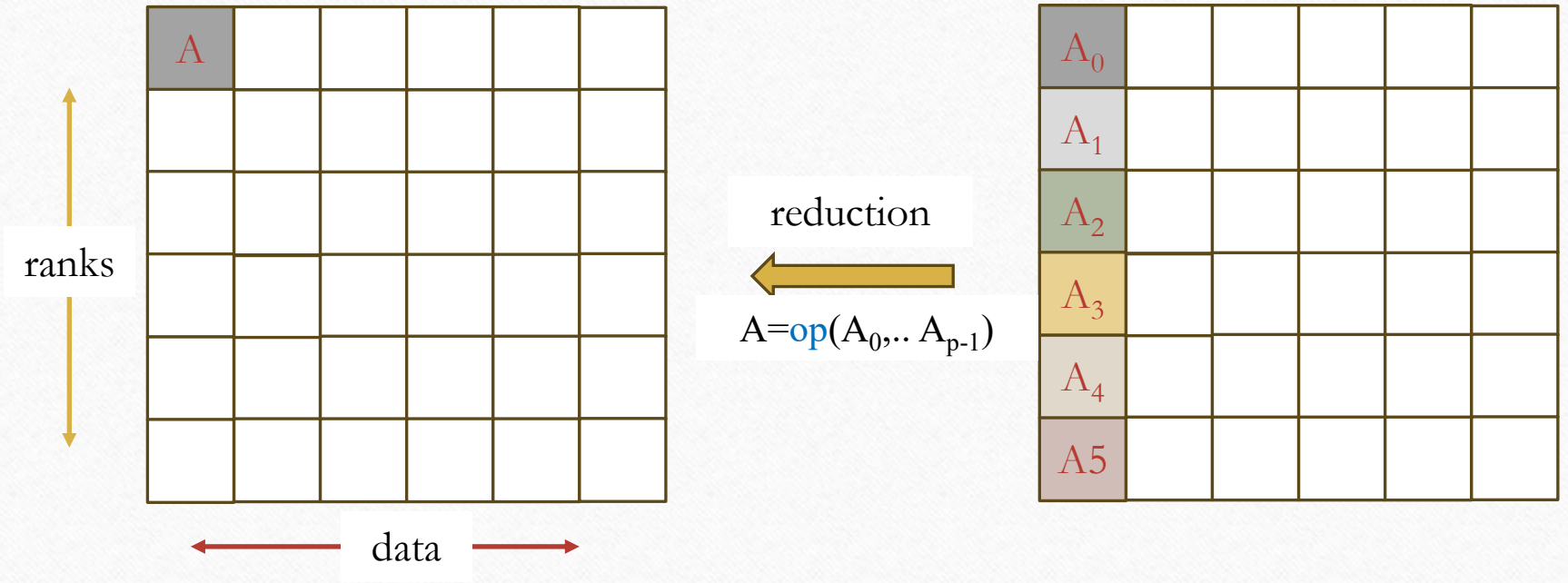
Broadcast

- int MPI_Bcast(void* buf, int count, MPI_Datatype dt, int root, MPI_Comm comm)
  - Each participant in a one-to-all broadcast calls the broadcast primitive (even though all but the root are receivers)
  - root is the rank that originates the data. The root is often rank 0, but any rank in the communicator can be root



Reduction

- int MPI_Reduce(const void* sndbuf, void* rcvbuf, int count, MPI_Datatype dt, MPI_Op op, int root , MPI_Comm comm)
  - Performs a reduction and place the result in one root process (all-to-one)



MPI_Op predefined Reduction operations

Name	Meaning
MPI_MAX	maximum
MPI_MIN	minimum
MPI_SUM	sum
MPI_PROD	product
MPI_LAND	logical and
MPI_BAND	bit-wise and
MPI_LOR	logical or
MPI_BOR	bit-wise or
MPI_LXOR	logical exclusive or (xor)
MPI_BXOR	bit-wise exclusive or (xor)
MPI_MAXLOC	max value and location
MPI_MINLOC	min value and location

- To use MPI_MINLOC/MPI_MAXLOC in a reduce operation the datatype argument must be a pair (value and location). MPI provides the following predefined datatypes:

Name	Description
MPI_FLOAT_INT	`float` and `int`
MPI_DOUBLE_INT	`double` and `int`
MPI_LONG_INT	`long` and `int`
MPI_2INT	pair of `int`
MPI_SHORT_INT	`short` and `int`
MPI_LONG_DOUBLE_INT	`long double` and `int`

MPI_MAXLOC/ MPI_MINLOC

values	19	21	15	28	19	28
indexes	0	1	2	3	4	5

MAXLOC(value, index) = (28,3)

MINLOC(value, index) = (15,2)

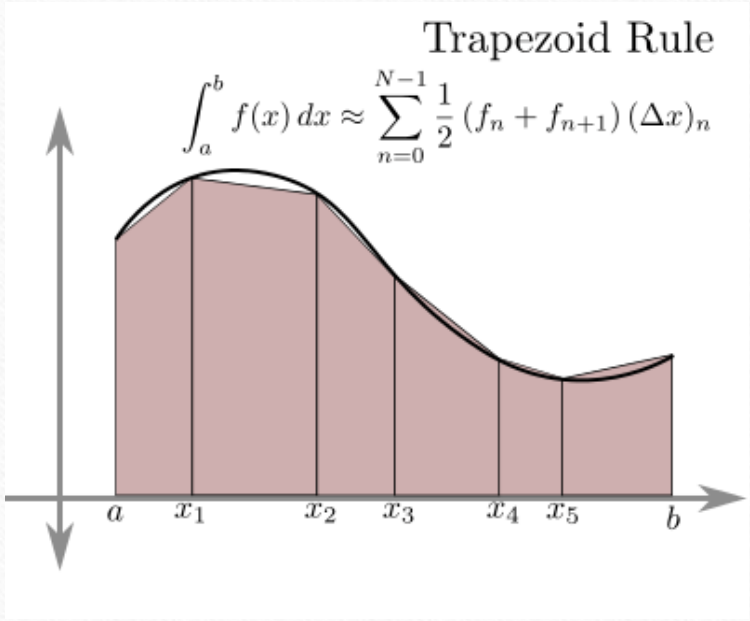
Tie-breaking rule: if multiple processes have the same min/max value, MPI_MINLOC/MPI_MAXLOC return the smallest index

```cpp
#include <cstdio>
#include <cstdlib>
#include <mpi.h>

int main(int argc, char *argv[]) {

    MPI_Init(&argc, &argv);
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    srandom(rank);

    // This corresponds to the predefined type MPI_2INT
    struct my2INT {
        int val;
        int idx;
    } in, out;

    in.val = random()%100;
    in.idx = rank;

    std::printf("Process %d val %d\n", in.idx, in.val);

    MPI_Reduce(&in, &out, 1, MPI_2INT, MPI_MAXLOC, 0, MPI_COMM_WORLD);

    // after the reduce, the maxloc values are in the root rank
    if (!rank) {
        std::printf("The maximum is %d with idx %d\n", out.val, out.idx);
    }

    MPI_Finalize();
    return 0;
}
```

Example: mpi_maxloc.cpp and mpi_maxminloc.cpp

MPI_Reduce example

- Trapezoid rule for numerical integration using MPI_Reduce()
  - The same example we already implemented using point-to-point communications

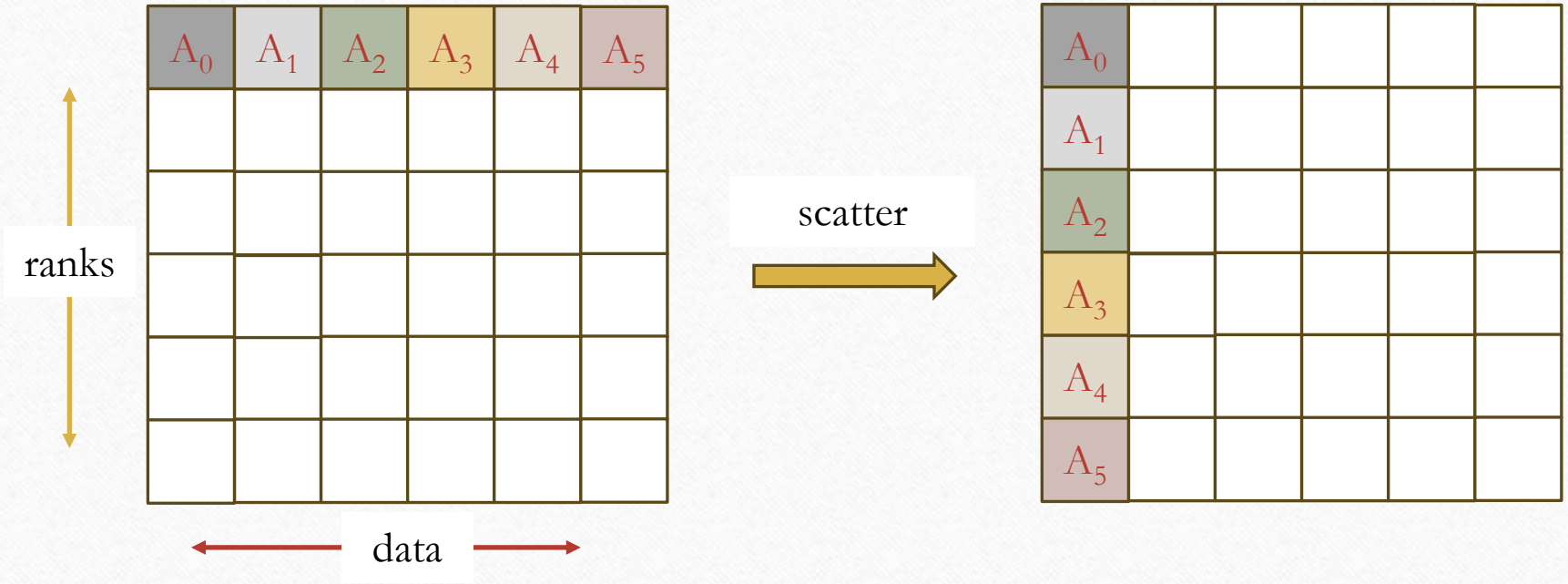



```
29  int main( int argc, char* argv[] ) {
30      const double a = 0.0;
31      const double b = 1.0;
32      const int    n = 10000000;
33      double partial_result;
34      double result = 0.0;
35      int myrank, size;
36
37      MPI_Init(&argc, &argv);
38      MPI_Comm_rank(MPI_COMM_WORLD, &myrank);
39      MPI_Comm_size(MPI_COMM_WORLD, &size);
40
41      // each node computes a partial result
42      partial_result = trap(myrank, size, a, b, n);
43      // global reduction, the final value stored in rank=0
44      MPI_Reduce(&partial_result, &result, 1, MPI_DOUBLE,
45                 MPI_SUM, 0, MPI_COMM_WORLD);
46
47      if (!myrank)
48          std::printf("Area: %.18f\n", result);
49
50      MPI_Finalize();
51      return 0;
52  }
```

Example: mpi_trapezoidreduce.cpp

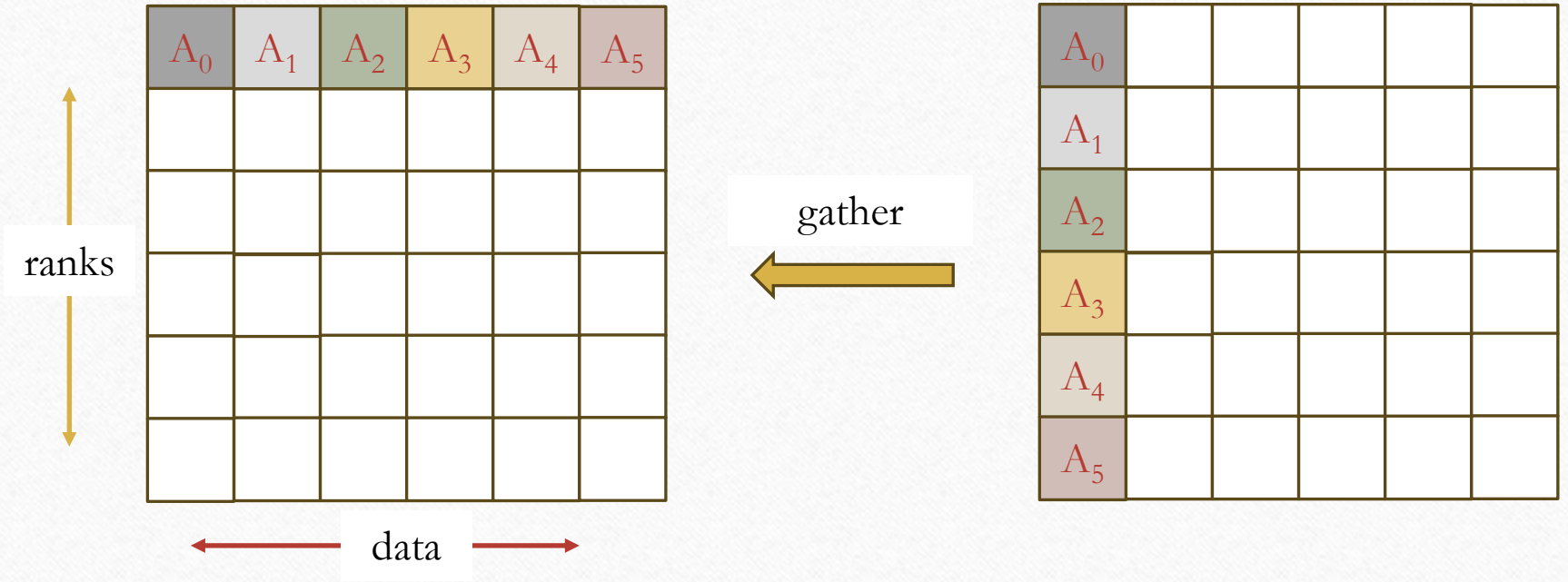
Scatter

- int MPI_Scatter(const void* sndbuf, int sndcount, MPI_Datatype snddt,
  void* rcvbuf, int rcvcount, MPI_Datatype rcvdt, int root , MPI_Comm comm)
  - Sends p data blocks from the root process to all other processes (including itself) in the communicator (one-to-all)



Gather

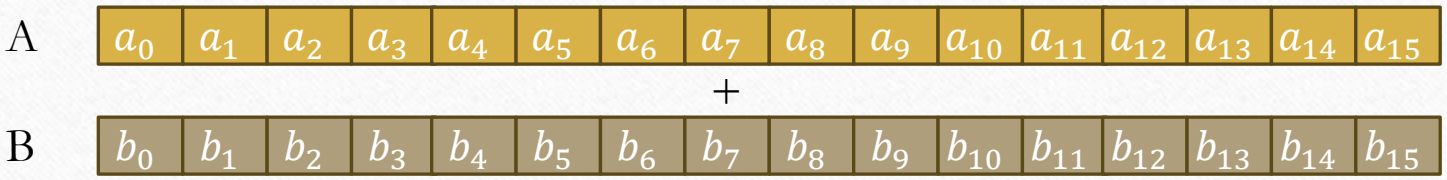
- int MPI_Gather(const void* sndbuf, int sndcount, MPI_Datatype snddt,
  void* rcvbuf, int rcvcount, MPI_Datatype rcvdt, int root , MPI_Comm comm)
  - The root receives p data blocks from all processes (including itself) in the communicator (all-to-one)



Map example: vector sum 1/2

- We want to execute a parallel map computation for the sum of two vectors of n elements

$$A, B, C \in \mathbb{R}^n \; c_i = a_i + b_i \text{ for all } i \in \{0, \ldots, n-1\}$$

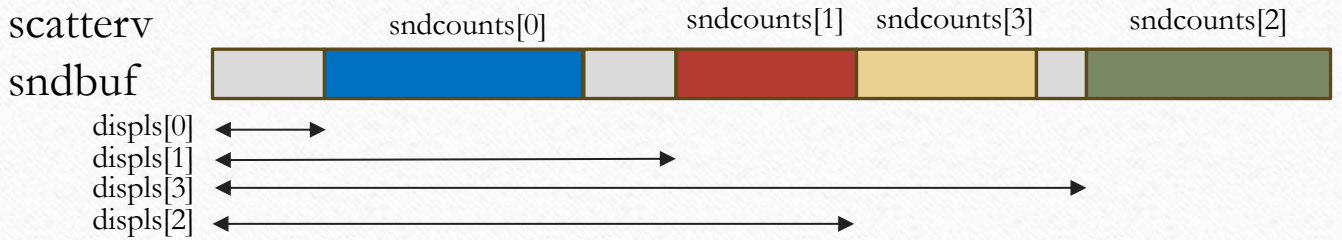


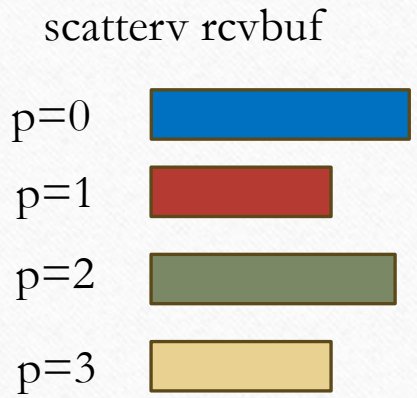
- With MPI using p processes: the root process partitions (Scatter operation) the A and B vectors to all p processes, each computing a local sum (i.e., a local C vector of about $\left\lfloor \frac{n}{p} \right\rfloor$ elements). Then, the root process collects all local vectors into the final result vector (Gather operation)
- In the first version $n \; mod \; p = 0$

Example: mpi_vectorsum.cpp

Map example: vector sum 2/2

- In this second version, n does not need to be a multiple of p. It also works when $n \ mod \ p \neq 0$
- int MPI_Scatterv(const void* sndbuf, const int sndcounts[], const int displs[], MPI_Datatype snddt,
  void* rcvbuf, int rcvcount, MPI_Datatype rcvdt, int root , MPI_Comm comm)
  int MPI_Gatherv(const void* sndbuf, int sndcount, MPI_Datatype sendtype,
  void* rcvbuf, const int rcvcounts[], const int displs[], MPI_Datatype rcvdt, int root , MPI_Comm comm)
  - Gaps are allowed between distinct partitions in the source data; partitions may have different message sizes, and the distribution order can be any



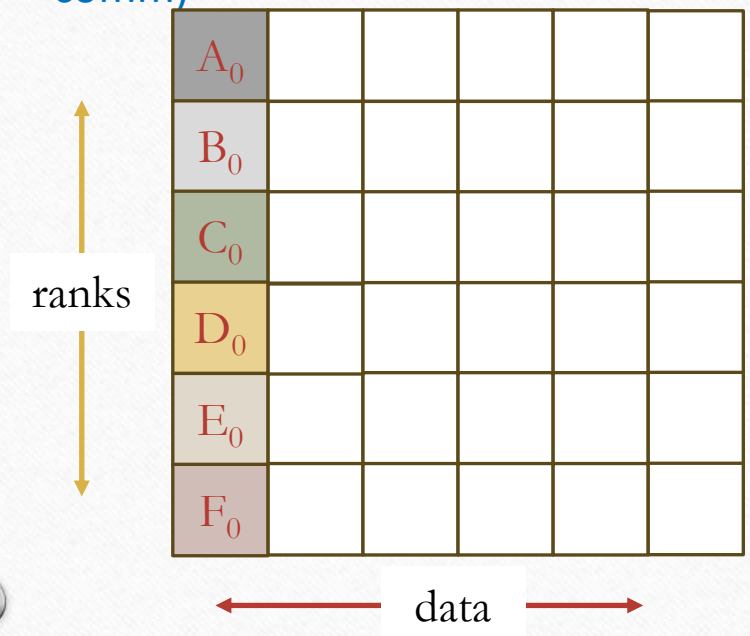


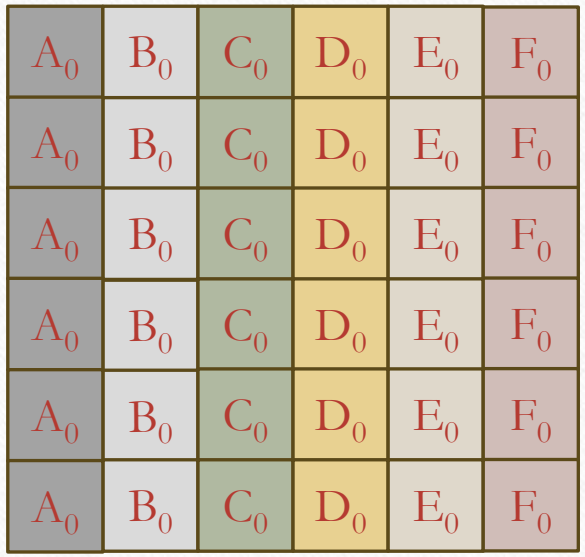
NOTE:
For Scatterv/Gatherv, displs entries are measured in extents of the corresponding datatype, not in bytes.
sndcounts[], displs[], recvcounts[] are significant only at root.

Example: mpi_vectorsumv.cpp

Allgather/Allgatherv

- int MPI_Allgather(const void* sndbuf, int sndcount, MPI_Datatype snddt,
  void* rcvbuf, int rcvcount, MPI_Datatype rcvdt, MPI_Comm comm)
  int MPI_Allgatherv(const void* sndbuf, int sndcount, MPI_Datatype snddt,
  void* rcvbuf, const int rcvcounts[], const int displs[], MPI_Datatype rcvdt, MPI_Comm comm)



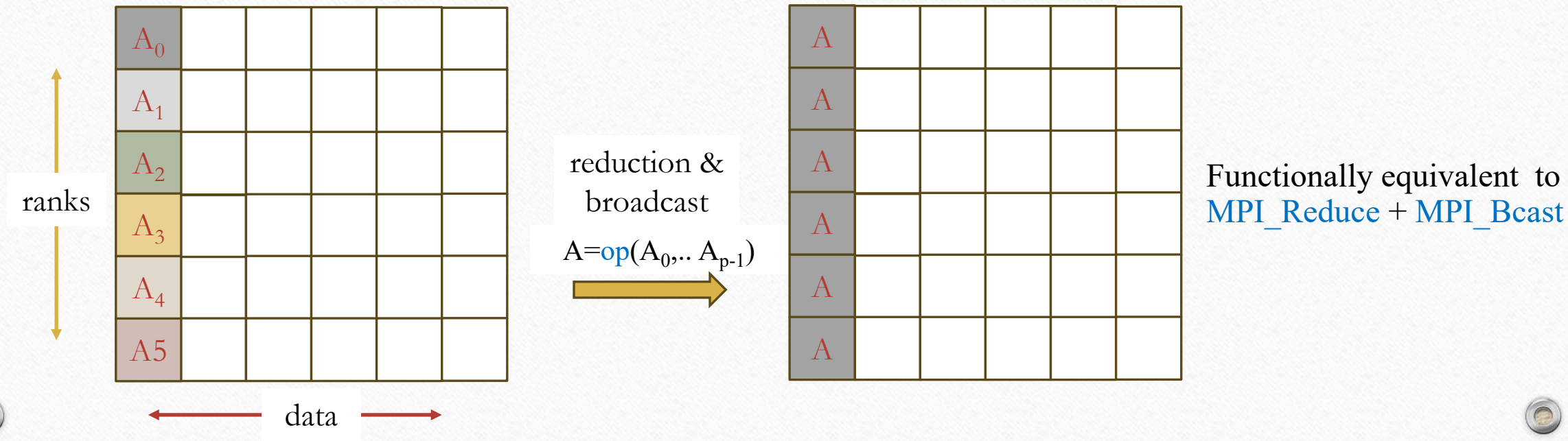


Functionally equivalent to MPI_Gather + MPI_Bcast

Example: mpi_collectreverse.cpp

Allreduce

- int MPI_Allreduce(const void* sndbuf, void* rcvbuf, int count, MPI_Datatype dt, MPI_Op op, MPI_Comm comm)



Functionally equivalent to MPI_Reduce + MPI_Bcast

Example: Power Iteration

- Power Iteration is one of the simplest numerical algorithms for computing a single eigenvalue-eigenvector pair , specifically the pair associated with the eigenvalue of largest magnitude (dominant eigenpair, i.e., λ_{max}, x_{max})
- The algorithm is sketched in the side box
- The parallelization follows a (iterative) Map-Reduce structure:
  - Partitioning A across processes and broadcasting the initial x_0
  - Each processor computes its local chunk $y = Ax$
  - MPI_Allgather/MPI_Allgatherv collects all local chunks of y
  - Normalize y to produce x_{k+1}
  - Compute λ through the Rayleigh quotient using the normalized vector
  - Check convergence $\|x_{k+1} - x_k\| < \varepsilon$
  - Norms, Rayleigh quotient, and convergence tests require global reductions, e.g., MPI_Allreduce

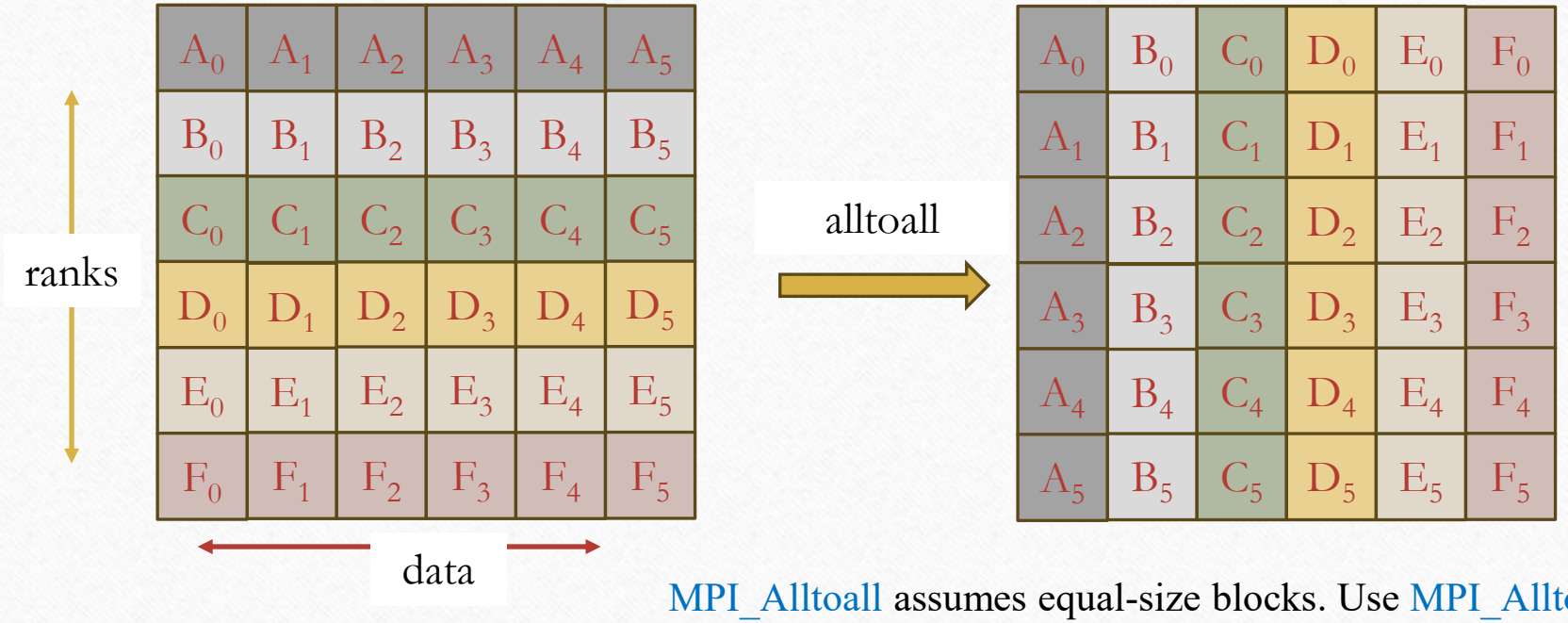
$A \in \mathbb{R}^{n \times n}$ diagonalizable, $x \in \mathbb{R}^n$

set $x_0 \neq 0$ at random
for k= 1,…, max_iters
do
$y_k = A\, x_{k-1}$ // matrix vector product
$x_k = \frac{y_k}{\|y_k\|_2}$ // normalization to ensure stability
$\lambda_k = \frac{x_k^T y_k}{x_k^T x_k}$ // Rayleigh quotient
if ($\|x_k - x_{k-1}\| < \varepsilon$) stop; // converged?
done

Examples: mpi_power-iteration_replicated.cpp and mpi_power-iteration_partitioned.cpp

Alltoall/Alltoallv

- int MPI_Alltoall(const void* sndbuf, int sndcount, MPI_Datatype snddt,
  void* rcvbuf, int rcvcount, MPI_Datatype rcvdt, MPI_Comm comm)
  int MPI_Alltoallv(const void* sndbuf, const int sndcounts[], const int sdispls[], MPI_Datatype snddt,
  void* rcvbuf, const int rcvcounts[], const int rdispls[], MPI_Datatype rcvdt, MPI_Comm comm)



A_0	A_1	A_2	A_3	A_4	A_5
B_0	B_1	B_2	B_3	B_4	B_5
C_0	C_1	C_2	C_3	C_4	C_5
D_0	D_1	D_2	D_3	D_4	D_5
E_0	E_1	E_2	E_3	E_4	E_5
F_0	F_1	F_2	F_3	F_4	F_5

A_0	B_0	C_0	D_0	E_0	F_0
A_1	B_1	C_1	D_1	E_1	F_1
A_2	B_2	C_2	D_2	E_2	F_2
A_3	B_3	C_3	D_3	E_3	F_3
A_4	B_4	C_4	D_4	E_4	F_4
A_5	B_5	C_5	D_5	E_5	F_5

Each process performs a scatter operation (analogous to a matrix transpose)

MPI_Alltoall assumes equal-size blocks. Use MPI_Alltoallv when block sizes differ

Example: distributed FFT

- FFT algorithms often require data redistribution between stages
- MPI_Alltoall implements the distributed transpose
- Each process splits its local coefficients into P blocks and sends one block to every other process
- After MPI_Alltoall, each process owns one frequency block collected from all processes
- The example uses the FFTW library for local FFTs
  - Production code should use the FFTW MPI interface or another tuned distributed FFT library
- The result is checked against a serial FFTW computation

Code notes:

- Input distribution: interleaved, rank r owns x[r], x[r+P], x[r+2P], ...
- Simplifying constraint: N % (P*P) == 0
- MPI_Alltoall is used with equal-size blocks
- --check compares the result with a serial FFTW computation
- Small numerical differences are expected for P > 1 due to floating-point ordering

Example: mpi_fft1d.cpp

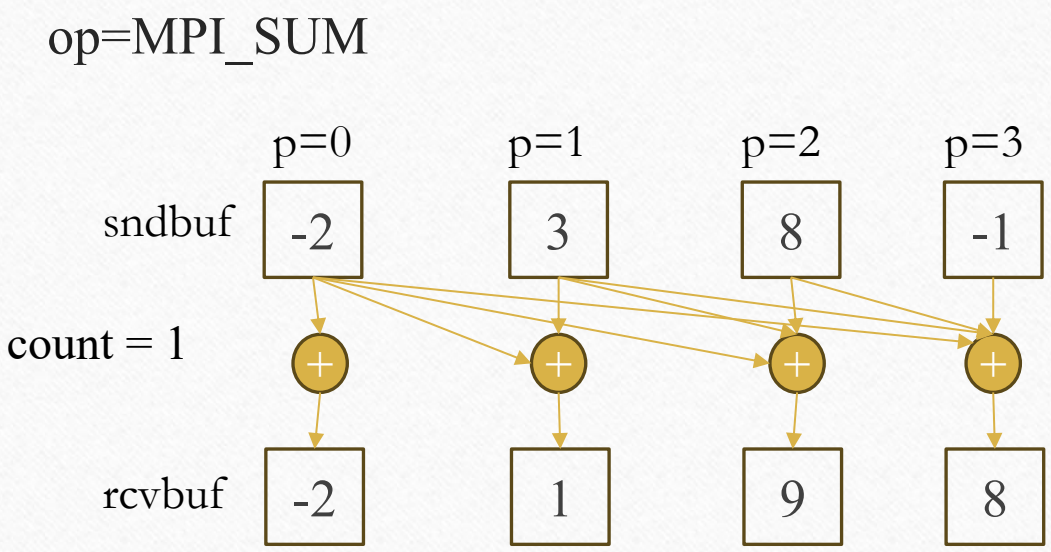
Parallel prefix operations

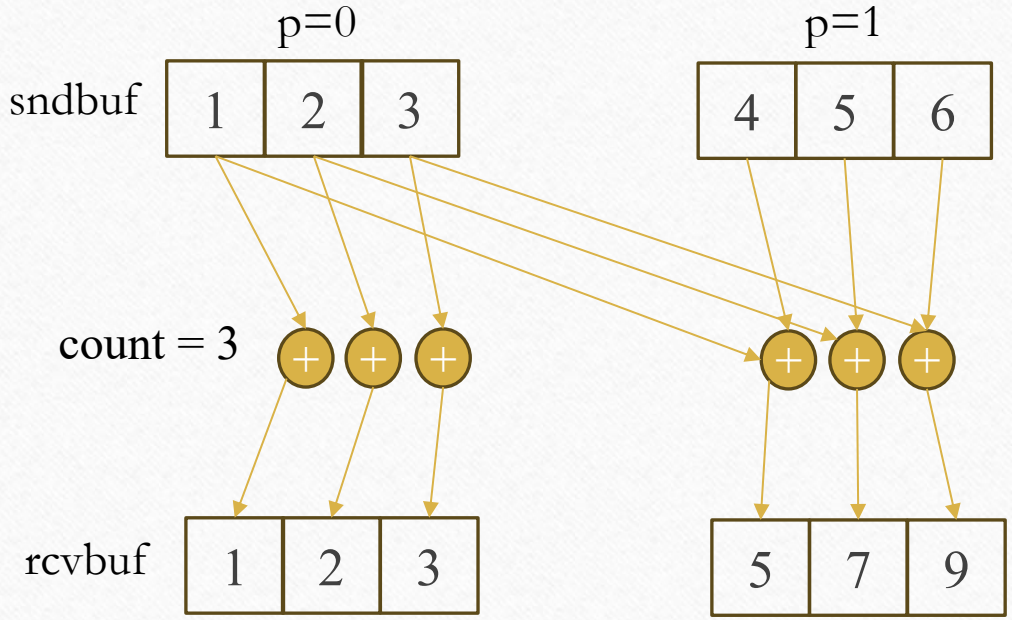
- int MPI_Scan(const void* sndbuf, void* rcvbuf, int count, MPI_Datatype dt, MPI_Op op, MPI_Comm comm)
  - Inclusive scan operation: process i compute $result = op(v_0, \ldots, v_i)$
- int MPI_Exscan(const void* sndbuf, void* rcvbuf, int count, MPI_Datatype dt, MPI_Op op, MPI_Comm comm)
  - Exclusive scan operation: process i compute $result = op(v_0, \ldots, v_{i-1})$

op = sum

Input vector	-2	4	2	-1	0	1	-3	2	0	4	1	5
scan	-2	2	4	3	3	4	1	3	3	7	8	13
exscan	undef	-2	2	4	3	3	4	1	3	3	7	8

MPI_Scan (inclusive)





For count = k, MPI_Scan performs k independent prefix reductions:
rcvbuf[j] = op(sndbuf_0[j], ..., sndbuf_rank[j]), j = 0, ..., k-1

Example: mpi_scan.cpp

List of core Collectives

Collectives	Pattern
MPI_Barrier	Synchronization
MPI_Bcast	One-to-all
MPI_Scatter	One-to-all blocks
MPI_Gather	All-to-one blocks
MPI_Reduce	All-to-one reduction
MPI_Allgather	All-to-all gather
MPI_Allreduce	All-to-all reduction
MPI_Alltoall	All-to-all personalized exchange
MPI_Scan	Inclusive prefix reduction
MPI_Exscan	Exclusive prefix reduction

Non-blocking variants :
MPI_Ibarrier, MPI_Ibcast, MPI_Iscatter, MPI_Igather, MPI_Ireduce, MPI_Iallgather, MPI_Iallreduce, MPI_Ialltoall, MPI_Iscan, ...

Variable-size variants :
MPI_Scatterv, MPI_Gatherv, MPI_Allgatherv, MPI_Alltoallv

Suggested Readings

- Chapter 9 of the «Parallel Programming Concepts and Practice» course book
- MPI: A Message-Passing Interface Standard Version 4.1
  - https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf
- Open MPI documentation:
  - https://docs.open-mpi.org/en/v5.0.x/
- MPI tutorial
  - https://hpc-tutorials.llnl.gov/mpi/
- OSU micro-benchmarks:
  - https://mvapich.cse.ohio-state.edu/benchmarks/
  - C benchmarks installed in /opt/ohpc/pub/mpi/osu-7.4/ on the spmcluster